

AWS Agent FinOps avec CrewIA

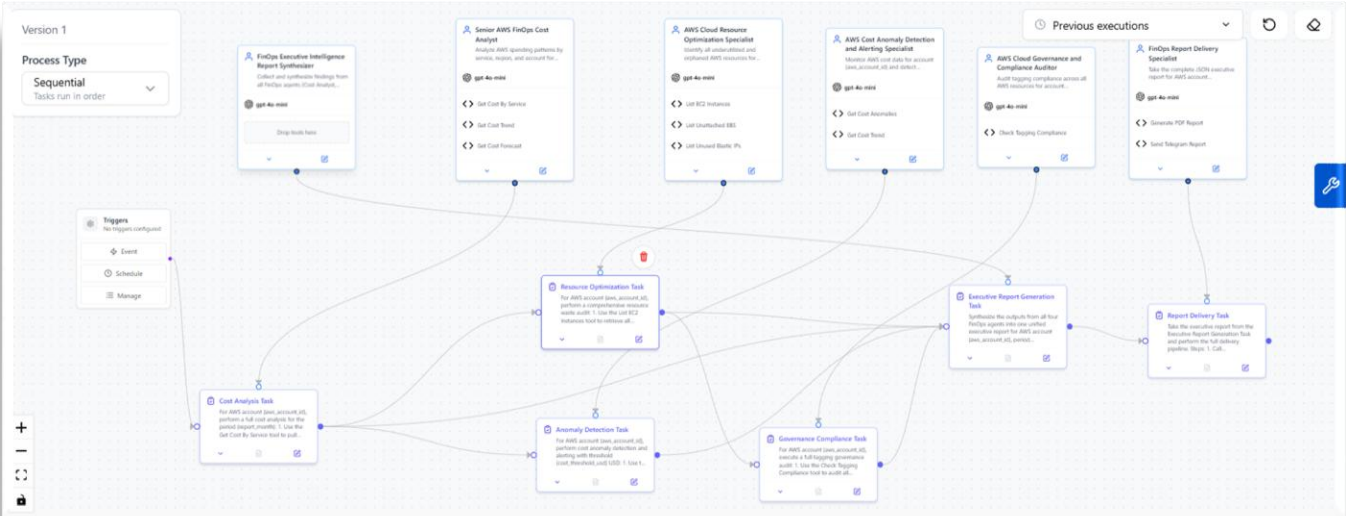
Réaliser par : Ayoub ELMORTAJI

Étudiant Ingénieur en Cybersécurité & Cloud Computing à ENSAM Casablanca

Vue d'ensemble

Le système AWS FinOps Agent est une chaîne de 6 agents IA spécialisés qui s'exécutent **en séquence**. Chaque agent réalise une tâche d'analyse précise et transmet son résultat à l'agent suivant, jusqu'à la livraison finale d'un rapport PDF exécutif envoyé directement sur Telegram.

L'objectif est d'automatiser de bout en bout le pilotage financier (FinOps) d'un compte AWS : analyse des coûts, détection des ressources gaspillées, détection d'anomalies, audit de conformité du tagging, puis synthèse et livraison d'un rapport prêt à l'emploi pour les équipes décisionnaires.



6	10	PDF + Telegram
agents IA spécialisés	outils custom (data + delivery)	livraison automatisée

Le flux complet

Le pipeline démarre lorsque l'automation est lancée avec trois paramètres d'entrée : l'identifiant du compte AWS, le mois du rapport et le seuil de coût pour la détection d'anomalies. Ces valeurs circulent ensuite à travers les six agents.

Lancement de l'automation — aws_account_id · report_month · cost_threshold

↓

1

Senior AWS FinOps Cost Analyst

Outils utilisés :

- get_cost_by_service → Cost Explorer (get_cost_and_usage, mensuel)
- get_cost_trend → Cost Explorer (quotidien, 30 jours)

- `get_cost_forecast` → Cost Explorer (prévision 30 jours)

Output : `cost_summary` — `total_cost`, `services[]`, `mom_change%`, `top_3_drivers`, `forecast_total`, `daily_trend_series`



2

AWS Cloud Resource Optimization Specialist

Outils utilisés :

- `list_ec2_instances` → EC2 + CloudWatch (CPU moyen)
- `list_unattached_ebs` → EC2 `describe_volumes` (`state=available`)
- `list_unused_elastic_ips` → EC2 `describe_addresses`

Output : `optimization_recommendations` — `recommendations[]`, `total_potential_savings_usd`



3

AWS Cost Anomaly Detection Specialist

Outils utilisés :

- `get_cost_anomalies` → Cost Explorer `get_anomalies`
- `get_cost_trend` → croisement avec les tendances de l'Agent 1

Output : `anomalies` — `alerts[]`, `severity` (`critical` / `warning` / `info`), `root_cause`



4

AWS Cloud Governance & Compliance Auditor

Outil utilisé :

- `check_tagging_compliance` → Resource Groups Tagging API

Output : `governance_score` — `score/100`, `non_compliant_resources[]`, `missing_tags`



5

FinOps Executive Intelligence Report Synthesizer

Outils : aucun — synthèse pure par l'IA à partir des 4 outputs précédents

- `executive_summary` (paragraphe narratif)
- `priority_actions[]` (triées par ROI)
- `total_potential_savings_usd`
- `chart_data` (compatible ngx-echarts Angular)

Output : `rapport JSON` unifié complet



6

FinOps Report Delivery Specialist**Outils utilisés :**

- generate_pdf_report → Python pur (io.BytesIO, PDF 1.4, sans dépendance externe)
- send_telegram_report → requests → Telegram Bot API

Output : confirmation de livraison + Message ID Telegram

Les 10 outils custom en détail

Outils de données AWS (8 outils boto3)

Outil	API AWS	Ce qu'il fait
get_cost_by_service	Cost Explorer get_cost_and_usage	Coûts du mois groupés par service (EC2, RDS, S3...) + variation MoM
get_cost_trend	Cost Explorer get_cost_and_usage	Coût quotidien sur 30 jours (granularité DAILY)
get_cost_forecast	Cost Explorer get_cost_forecast	Prévision des 30 prochains jours (confiance 85%)
list_ec2_instances	EC2 + CloudWatch	Liste toutes les instances + CPU moyen sur 7 jours
list_unattached_ebs	EC2 describe_volumes	Volumes EBS non attachés (state = available)
list_unused_elastic_ips	EC2 describe_addresses	EIPs sans AssociationId = gaspillage à 3,65 \$/mois
check_tagging_compliance	Resource Groups Tagging API	Audit des tags Owner / Project / Environment sur toutes les ressources
get_cost_anomalies	Cost Explorer get_anomalies	Anomalies de coût sur 90 jours au-dessus du seuil défini

Outils de livraison (2 outils)

Outil	Technologie	Ce qu'il fait
generate_pdf_report	Python pur (io.BytesIO)	Génère un PDF valide (PDF 1.4) avec toutes les sections, sans dépendance externe
send_telegram_report	requests	Lit le PDF depuis /tmp/ et l'envoi via l'API Telegram Bot

Comment l'IA prend ses décisions

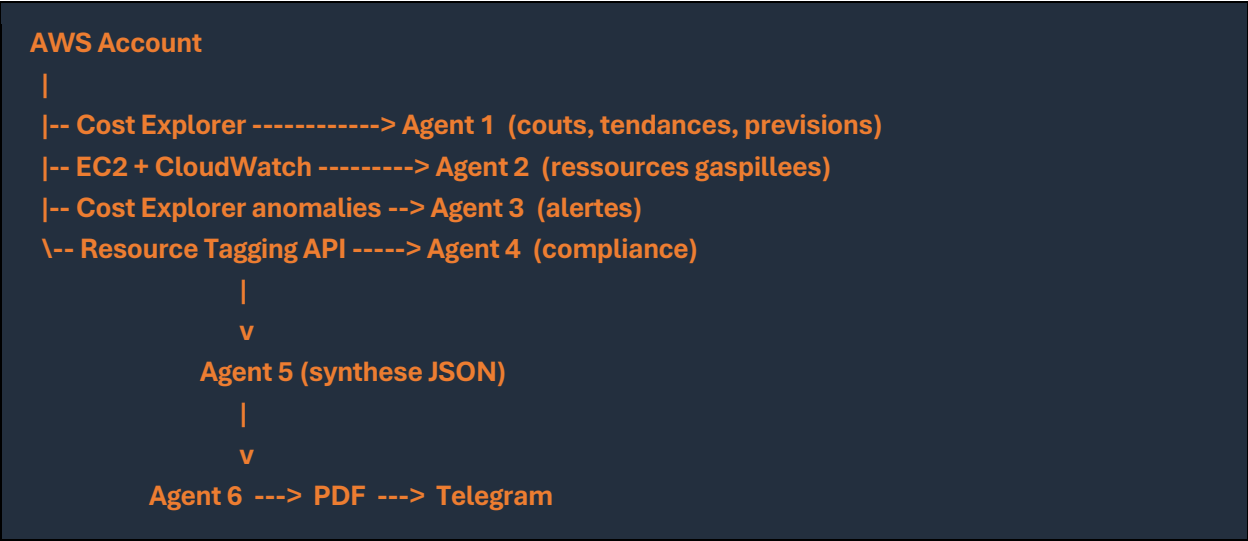
Chaque agent reçoit cinq éléments pour opérer de façon autonome :

- Son rôle — qui il est

- Son goal — ce qu'il doit produire
- Sa backstory — son domaine d'expertise
- Le contexte — les outputs des agents précédents
- Ses outils — les API qu'il est autorisé à appeler

À partir de ces éléments, l'agent décide lui-même quels outils appeler, dans quel ordre, et comment interpréter les résultats pour produire un output structuré et exploitable par l'agent suivant.

Flux des données



Variables d'entrée

Variable	Exemple	Usage
{aws_account_id}	---	Transmis à tous les outils boto3
{report_month}	2025-06	Définit la plage de dates pour Cost Explorer
{cost_threshold_usd}	1000	Seuil minimum pour la détection d'anomalies

Authentification AWS

Tous les outils utilisent le même pattern d'authentification :

```
session = boto3.Session(  
    aws_access_key_id = os.environ.get("AWS_ACCESS_KEY_ID"),  
    aws_secret_access_key = os.environ.get("AWS_SECRET_ACCESS_KEY"),  
    region_name      = os.environ.get("AWS_DEFAULT_REGION", "us-east-1")  
)
```

Note : J'ai créer un user IAM (**finops-agent**) avec les droits :

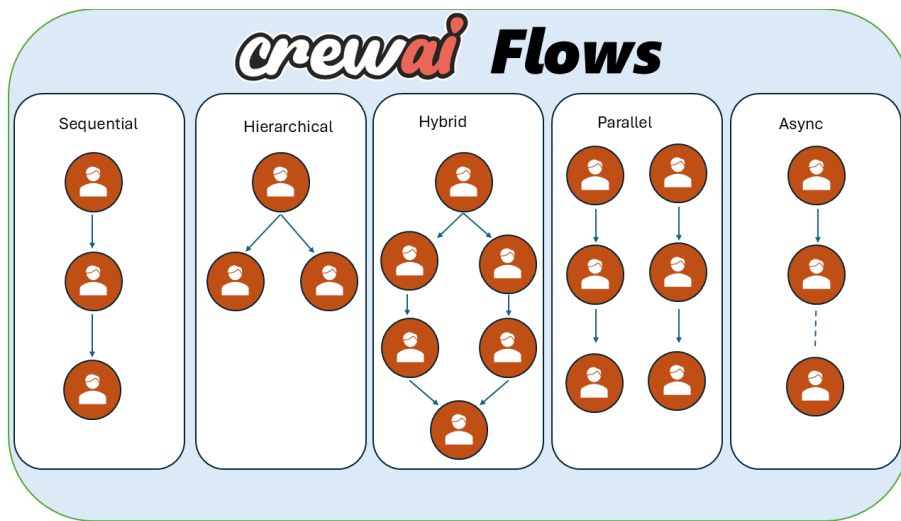
Permissions policies (2)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type All types < 1 > ⚙️

☐	Policy name ↗	Type	Attached via ↗
☐	 AWSBillingReadOnlyAccess	AWS managed	Directly
☐	 ReadOnlyAccess	AWS managed - job function	Directly

Pourquoi c'est séquentiel et pas parallèle ?



Chaque agent a besoin du contexte des agents précédents pour enrichir son analyse. C'est une chaîne de raisonnement, et non une somme de tâches indépendantes. Par exemple :

- L'Agent 5 a besoin des 4 outputs précédents pour construire un rapport cohérent.
- L'Agent 3 (anomalies) se base sur les tendances calculées par l'Agent 1 pour croiser les données.
- L'Agent 2 enrichit ses recommandations d'optimisation avec le contexte des coûts fourni par l'Agent 1.

Fiche détaillé sur les 6 agents :

1

AGENT 1

Senior AWS FinOps Cost Analyst

GOAL

Analyser les patterns de dépenses AWS par service, région et compte pour le compte {aws_account_id} durant {report_month}. Détecter les anomalies de coût (>20% de variation), comparer le mois courant au mois précédent et prévoir les coûts des 30 prochains jours. Produire un bloc JSON cost_summary structuré identifiant les 3 principaux facteurs de coût et signalant tout service en hausse de plus de 20% en MoM.

BACKSTORY

Praticien FinOps AWS chevronné avec plus de 10 ans d'expérience dans la gestion des coûts cloud pour des entreprises du Fortune 500. Expertise approfondie de Cost Explorer, des API de facturation et des stratégies d'allocation des coûts. Méthodique et orienté données, reconnu pour faire émerger des insights exploitables à partir de données brutes. Maîtrise les nuances des modèles de tarification EC2, des Reserved Instances, des Savings Plans, et de la façon dont des événements de scaling imprévus peuvent provoquer des pics de coûts. Ses rapports sont toujours structurés, précis et prêts pour une consommation exécutive.

OUTILS (3)

Outil	API AWS réelle
Get Cost By Service	boto3 Cost Explorer get_cost_and_usage — MONTHLY, groupé par SERVICE
Get Cost Trend	boto3 Cost Explorer get_cost_and_usage — DAILY, 30 derniers jours
Get Cost Forecast	boto3 Cost Explorer get_cost_forecast — 30 jours à venir, confidence 85%

TÂCHE : COST ANALYSIS TASK

- Appel Get Cost By Service → coûts mensuels par service, calcul MoM, flag tout service avec >20% d'augmentation.
- Appel Get Cost Trend → tableau quotidien 30 jours, identifier pic et creux.
- Appel Get Cost Forecast → projection 30 jours, total et moyenne quotidienne.

4. Synthèse en JSON cost_summary avec daily_trend_series compatible ngx-echarts.

Contexte reçu : aucun (premier agent)

OUTPUT ATTENDU

```
{
  "cost_summary": {
    "total_cost_usd": 28322.15,
    "previous_month_total_usd": 24410.0,
    "mom_change_pct": 16.0,
    "services": [ { "service": "EC2", "cost_usd": 12430.5, "mom_change_pct": 21.9 }, ... ],
    "flagged_services": [ { "service_name": "EKS", "change_pct": 47.4 } ],
    "top_3_cost_drivers": [ { "service": "EC2", "cost_usd": 12430.5 }, ... ],
    "forecast_total_30d_usd": 31627.0,
    "forecast_avg_daily_usd": 1054.23,
    "daily_trend_series": { "xAxis": [ "2025-06-01", ... ], "yAxis": [ 923.40, ... ] },
    "daily_forecast_series": { "xAxis": [ ... ], "yAxis": [ ... ], "lower_bound": [ ... ], "upper_bound": [ ... ] }
  }
}
```

2

AGENT 2

AWS Cloud Resource Optimization Specialist

GOAL

Identifier toutes les ressources AWS sous-utilisées et orphelines pour le compte {aws_account_id}. Signaler les instances EC2 avec une utilisation CPU inférieure à 10%, découvrir les volumes EBS non attachés et localiser les Elastic IPs inutilisées. Calculer précisément le potentiel d'économies mensuelles pour chaque constat et produire un bloc JSON optimization_recommendations trié par potentiel d'économies décroissant.

BACKSTORY

Architecte cloud AWS spécialisé dans l'optimisation des coûts d'infrastructure. A aidé des entreprises à réduire leur facture AWS de 35% en moyenne en identifiant le gaspillage — instances inactives, volumes oubliés, ressources réseau orphelines. Sait exactement quels signaux indiquent un surprovisionnement et repère avec précision les ressources qui étaient nécessaires autrefois mais qui ne font aujourd'hui que consommer du budget. Ses recommandations sont toujours spécifiques, actionnables et chiffrées en dollars.

OUTILS (3)

Outil	API AWS réelle
List EC2 Instances	boto3 EC2 describe_instances + CloudWatch get_metric_statistics (CPU 7j)
List Unattached EBS	boto3 EC2 describe_volumes avec filtre state=available
List Unused Elastic IPs	boto3 EC2 describe_addresses — filtre : pas d'AssociationId

TÂCHE : RESOURCE OPTIMIZATION TASK

1. List EC2 Instances → filtrer CPU < 10% → candidats à right-sizing ou suppression.
2. List Unattached EBS → chaque volume = 100% gaspillage.
3. List Unused Elastic IPs → 3,65 \$/mois chacune.
4. Produire des recommandations triées par savings décroissant.
5. Inclure savings_by_type_chart (pie chart EC2/EBS/EIP pour ngx-echarts).

Contexte reçu : output de Cost Analysis Task

OUTPUT ATTENDU

```
{
  "optimization_recommendations": {
    "total_potential_savings_usd": 1616.21,
    "findings_count": 14,
    "recommendations": [
      {
        "resource_id": "i-0c3d4e5f6a7b",
        "resource_type": "EC2 Instance",
        "issue_description": "CPU utilization 3.1%",
        "estimated_monthly_saving_usd": 560.64,
        "recommended_action": "Terminate or right-size",
        "region": "us-east-1",
        "priority": "high"
      }, ...
    ],
    "savings_by_type_chart": [
      { "name": "EC2", "value": 1285.96 },
      { "name": "EBS", "value": 312.00 },
      { "name": "EIP", "value": 18.25 }
    ]
  }
}
```

```
]
}
}
```

GOAL

Surveiller les données de coûts AWS pour le compte {aws_account_id} et détecter les pics ou baisses de coûts inattendus. Classifier chaque anomalie par sévérité — critical (>50% de variation), warning (20–50%) ou info (<20%). Identifier le service responsable et la fenêtre temporelle, puis générer des objets d'alerte structurés avec des actions de remédiation recommandées, prêts pour un affichage en dashboard.

BACKSTORY

Expert en intelligence financière cloud avec un parcours combinant DevOps et gestion des coûts. A construit des pipelines de détection d'anomalies pour de grands environnements AWS et sait qu'un pic de coût détecté tôt peut faire économiser des dizaines de milliers de dollars. Rigoureux sur la classification exacte de la sévérité — une alerte critique réveille les équipes en pleine nuit, donc jamais de fausse alerte. Chaque alerte générée inclut une hypothèse de cause racine et une action recommandée concrète.

OUTILS (2)

Outil	API AWS réelle
Get Cost Anomalies	boto3 Cost Explorer get_anomalies — 90j lookback, filtre par TotalImpact >= threshold
Get Cost Trend	boto3 Cost Explorer get_cost_and_usage — pour cross-référencer les pics quotidiens

TÂCHE : ANOMALY DETECTION TASK

1. Get Cost Anomalies avec le seuil {cost_threshold_usd}.
2. Get Cost Trend pour cross-référencer la timeline des anomalies.
3. Classifier la sévérité : critical (|%| >= 40%), warning (>= 5%), info (< 5%).
4. Enrichir chaque anomalie avec root cause et recommended_action.
5. Produire severity_chart_data pour ngx-echarts (bar chart).

Contexte reçu : output de Cost Analysis Task

OUTPUT ATTENDU

```
{
  "anomalies": {
    "total_anomalies": 6,
    "critical_count": 2,
    "warning_count": 3,
    "info_count": 1,
    "total_excess_cost_usd": 4030.55,
    "alerts": [
      {
        "anomaly_id": "anomaly-001",
        "service": "EKS",
        "region": "us-east-1",
        "severity": "critical",
        "percentage_change": 47.4,
        "estimated_excess_cost_usd": 1800.0,
        "root_cause": "Unexpected node group scaling event",
        "detected_at": "2025-06-18T14:32:00Z",
        "recommended_action": "Review EKS auto-scaling policies"
      }, ...
    ]
  }
}
```

```
},  
"severity_chart_data": {  
  "xAxis": ["Critical", "Warning", "Info"],  
  "series": [2, 3, 1]  
}  
}  
}
```

4

AGENT 4

AWS Cloud Governance and Compliance Auditor

GOAL

Auditer la conformité du tagging sur toutes les ressources AWS pour le compte {aws_account_id}. Vérifier que toutes les ressources portent les tags requis Owner, Project et Environment. Quantifier la non-conformité avec un governance score (0–100), identifier les principaux services et équipes en infraction, et produire un bloc JSON governance_score avec une liste de remédiation priorisée.

BACKSTORY

Spécialiste de la gouvernance cloud ayant mis en œuvre des politiques de tagging et des frameworks d'allocation des coûts pour des AWS Organizations multi-comptes. Sait qu'un mauvais tagging est la cause racine des problèmes d'attribution de facturation, des failles de sécurité et des audits échoués. Allie la rigueur d'un auditeur de conformité au pragmatisme d'un ingénieur — ses rapports exposent les violations tout en les priorisant par impact financier pour que les équipes sachent exactement par où commencer.

OUTILS (1)

Outil	API AWS réelle
Check Tagging Compliance	boto3 resourcegroupstaggingapi get_resources — paginé, vérifie Owner/Project/Environment

TÂCHE : GOVERNANCE COMPLIANCE TASK

1. Check Tagging Compliance → audit de toutes les ressources pour Owner, Project, Environment.
2. Comptabiliser compliant vs non-compliant.
3. Calculer le score = (compliant / total) * 100.
4. Identifier les principaux services violateurs par nombre de non-conformités.
5. Construire remediation_priority_list triée par coût mensuel.
6. Produire compliance_donut_chart_data pour ngx-echarts.

Contexte reçu : output de Resource Optimization Task

OUTPUT ATTENDU

```
{
  "governance_score": {
    "overall_score": 62.2,
    "total_resources_audited": 45,
    "compliant_count": 28,
    "non_compliant_count": 17,
    "missing_tags_breakdown": { "Owner": 8, "Project": 6, "Environment": 10 },
    "top_violating_services": [
      { "service_name": "EC2", "non_compliant_count": 5, "total_count": 12, "compliance_pct": 58.3 },
      ...
    ],
    "remediation_priority_list": [ ... ],
    "compliance_donut_chart_data": [
      { "name": "Compliant", "value": 28 },
      { "name": "Non-Compliant", "value": 17 }
    ]
  }
}
```

GOAL

Collecter et synthétiser les résultats de tous les agents FinOps (Cost Analyst, Resource Optimizer, Anomaly Detector, Governance Auditor) pour le compte AWS {aws_account_id} durant {report_month}. Produire un rapport JSON unique, unifié et bien formé, optimisé pour une consommation par dashboard Angular avec des structures de données compatibles ngx-echarts. Trier priority_actions par ROI décroissant. Inclure un executive_summary narratif convaincant.

BACKSTORY

Expert en reporting FinOps spécialisé dans la traduction de données de coûts cloud complexes en intelligence exécutive claire et actionnable. A fait le pont entre les équipes d'ingénierie et les décideurs du C-suite, en concevant des dashboards et rapports qui pilotent de vraies décisions business. Meticuleux sur la structure JSON, s'assurant toujours que les données sont formatées pour être directement consommées par des bibliothèques de graphiques frontend comme ngx-echarts. Sait qu'un bon rapport raconte une histoire — il commence par les chiffres clés, met en avant les plus grandes opportunités, et se termine par un appel à l'action clair.

OUTILS

Aucun — cet agent utilise uniquement son raisonnement LLM pour synthétiser les 4 outputs précédents.

TÂCHE : EXECUTIVE REPORT GENERATION TASK

Collecte les 4 outputs (cost_summary, optimization_recommendations, anomalies, governance_score) puis :

1. Écrire executive_summary (3-5 phrases narratives).
2. Calculer total_potential_savings_usd = optimisation + anomalies.
3. Construire priority_actions (top 5-8, triées par impact_usd décroissant).
4. Ajouter report_metadata (timestamp, account_id, period).
5. IMPORTANT : exclure les grands arrays (daily_trend_series, chart_data, etc.) pour garder l'output < 6000 caractères.

Contexte reçu : outputs des 4 tâches précédentes

OUTPUT ATTENDU

```
{
  "report_metadata": { "generated_at": "2026-07-20T12:00:00Z", "account_id": "...", "period": "2026-07", "report_version": "1.0" },
  "executive_summary": "In July 2026, total AWS spend reached $28,322.15...",
  "cost_summary": { ... },
  "anomalies": { ... },
  "optimization_recommendations": { ... },
  "governance_score": { ... },
  "total_potential_savings_usd": 4349.27,
  "priority_actions": [
    { "action": "Implement IAM boundaries for EC2", "impact_usd": 2230.5, "effort": "medium", "category": "anomaly", "timeframe": "immediate" },
    ...
  ]
}
```

GOAL

Prendre le rapport JSON exécutif complet pour le compte AWS {aws_account_id} ({report_month}), en générer un document PDF professionnel et le livrer instantanément aux parties prenantes via Telegram. Confirmer la livraison réussie avec l'ID du message Telegram.

BACKSTORY

Responsable du dernier kilomètre du reporting FinOps — transformer une analyse générée par IA en documents PDF soignés, prêts pour un comité de direction, et les livrer instantanément aux bonnes personnes. A construit des pipelines de reporting pour des équipes cloud du Fortune 500, et sait qu'un excellent insight ne vaut rien s'il n'atteint jamais un décideur. Meticuleux sur la qualité de présentation et la confirmation de livraison — ne considère jamais une tâche terminée tant qu'il n'a pas la preuve que le destinataire a reçu le rapport.

OUTILS (2)

Outil	Technologie
Generate PDF Report	Python pur (io.BytesIO) — PDF 1.4 from scratch, 6 sections, 2 pages A4
Send Telegram Report	requests → Telegram Bot API sendDocument — lit le PDF depuis /tmp/

TÂCHE : REPORT DELIVERY TASK

1. Appel generate_pdf_report avec le JSON strippé (sans les arrays volumineux) → retourne { pdf_path, period, total_cost_usd, total_savings_usd }.
2. Extraire les 4 valeurs individuelles du résultat.
3. Appel send_telegram_report avec ces 4 paramètres individuels (le tool lit le PDF directement depuis le disque).
4. Reporter le résultat avec le Message ID Telegram.

Contexte reçu : output de Executive Report Generation Task

OUTPUT ATTENDU

```
PDF Generation: /tmp/finops_report.pdf      [OK]
Telegram Delivery: success — Message ID: 3  [OK]
```

```
Report: finops_report_2026-07.pdf
Period: 2026-07 | Total Spend: $28,322.15 | Savings: $4,349.27
```

Récapitulatif visuel

Vue d'ensemble des dépendances entre agents : outils mobilisés, contexte reçu en entrée et agent(s) dont chacun dépend.

Agent	Outils	Contexte reçu	Dépend de
1. Cost Analyst	3	Aucun (1er agent)	—
2. Optimizer	3	Cost Analysis Task	Agent 1
3. Anomaly Det.	2	Cost Analysis Task	Agent 1
4. Governance	1	Resource Optimization Task	Agent 2
5. Synthesizer	0	Tâches 1 + 2 + 3 + 4	Agents 1, 2, 3, 4
6. Delivery	2	Executive Report Task	Agent 5

TOTAL : 11 outils mobilisés à travers les 6 agents.

Annexe :

